§1 Setup & Tooling

Summary
The three tools the rest of the ladder assumes are working and cross-checked. **TransformerLens** caches all 210 activations of GPT-2 small, and the final logits can be rebuilt exactly from the cached residual stream (max error 0.0). **nnsight** reads and edits the same activations through a model-agnostic tracing API that can also run remotely on NDIF. **SAELens** loads a pretrained sparse autoencoder whose features are readable at a glance (" Golden", " San", " Francisco"). It explains 87% of the residual variance on prose but only 65% on code.

1 Environment

Everything runs from one `uv` project (Python 3.11, PyTorch 2.14, TransformerLens 3.9, SAELens 6.51, nnsight 0.7). TransformerLens is pinned below 4.0: version 4 removes `HookedTransformer`, which ARENA, YSDA and SAELens still use. All of §1 runs on a laptop CPU in under a minute per script. Section 2 onwards can offload to Colab or Kaggle GPUs through the MCP servers set up by `scripts/setup_gpu_mcp.sh` (see the README). As a cross-check, the SAE script gave identical numbers on a Colab T4.

```
uv sync
uv run python experiments/00_setup/01_transformerlens_first_pass.py
uv run python experiments/00_setup/02_nnsight_activations.py
uv run python experiments/00_setup/03_saelens_pretrained_sae.py
uv run python scripts/build_writeups.py 01
```

2 TransformerLens: the residual stream

A transformer with L layers keeps a **residual stream** $x_\ell \in \mathbb{R}^{d_{\text{model}}}$ at every position. Each attention layer and MLP reads from it and adds its output back:

$$x_0 = W_E t + W_{\text{pos}}, \quad x_{\ell+1} = x_\ell + \sum_{h=1}^{H} \text{attn}_{\ell,h}(x_\ell) + \text{mlp}_\ell\left(x_\ell + \sum_h \text{attn}_{\ell,h}(x_\ell)\right), \tag{1}$$

and the logits are $W_U\ \text{LN}(x_L)$. Because the stream is a sum, the output of every component lives in the same space, which is what makes it possible to attribute behaviour to individual heads.

Sanity check. On the IOI prompt *"When Mary and John went to the store, John gave a drink to"* we run GPT-2 small ($L = 12$, $H = 12$, $d_{\text{model}} = 768$) once with `run_with_cache`. Recomputing $W_U\ \text{LN}(x_L)$ from the cached `resid_post` of the last layer reproduces the forward-pass logits to within 0.0. The model predicts `Mary` (0.68), `them` (0.13), `the` (0.05).

Logit lens. Applying the unembedding early, $\text{logits}_\ell = W_U\ \text{LN}(x_\ell)$, shows when the answer becomes linearly readable. The rank of " Mary" falls from 43262 at the embeddings to 276 before layer 8 and 22 before layer 9. It reaches rank 0 (top prediction) before layer 10. This matches the IOI circuit (Wang et al., 2022), whose name-mover heads sit in layers 9–10. Head 9.9, one such name mover, puts 0.67 of its final-position attention on `Mary` (Figure 1, right).

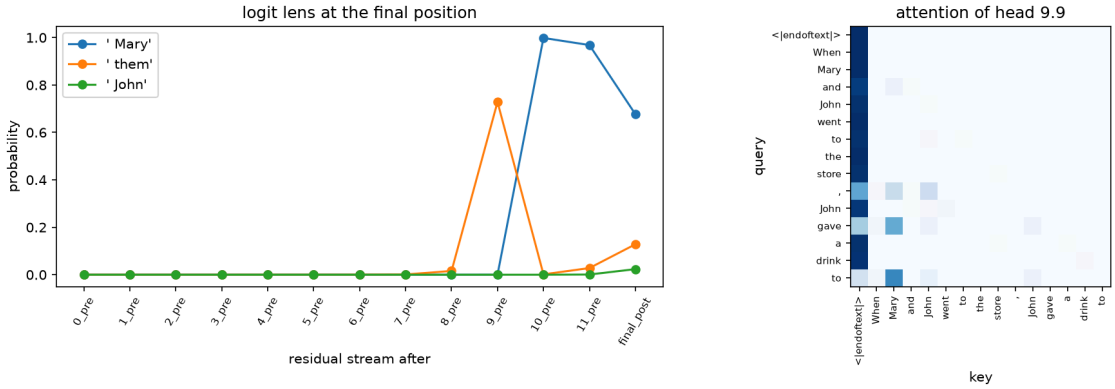


Figure 1: Left: logit-lens probability at the final position. Before layer 9 the model backs the generic " them" (0.73 after layer 8). The indirect object " Mary" takes over only after layers 9–10, and the subject " John" is kept near zero throughout. Right: attention pattern of head 9.9. The final query (" to") attends to " Mary".

3 nnsight: tracing and intervening

nnsight wraps a HuggingFace model and records a computation graph inside a `with model.trace(prompt):` block. Reading `.output` of any module and calling `.save()` returns the tensor. **Assigning** to it edits the forward pass. The same code runs remotely on NDIF with `remote=True`, where only the saved tensors are returned. That is the route to activation access on models too large for local hardware.

On *"The Eiffel Tower is located in the city of"*, GPT-2 puts 0.064 on " Paris". Zero-ablating the layer-6 MLP (setting its output to 0) **raises** this to 0.085.

> **Caveat**
> Zero-ablation moves activations off-distribution, because an MLP output of exactly 0 never occurs in training. A single-layer ablation that "helps" is therefore weak evidence. It could be removing an inhibitory contribution, or an artefact of the edit itself. Mean-ablation, or resample-ablation from other prompts, are the better defaults. Both appear in §2.

4 SAELens: a pretrained sparse autoencoder

A sparse autoencoder (SAE) rewrites an activation $x \in \mathbb{R}^{768}$ as a sparse, non-negative combination of $d_{\text{sae}} = 24576$ learned directions:

$$f(x) = \text{ReLU}(W_{\text{enc}}(x - b_{\text{dec}}) + b_{\text{enc}}), \quad \hat{x} = W_{\text{dec}} f(x) + b_{\text{dec}}, \tag{2}$$

trained to minimise $\|x - \hat{x}\|_2^2 + \lambda \|f(x)\|_1$. The hope is that each coordinate of f is a **monosemantic feature**. We use `gpt2-small-res-jb` at `blocks.8.hook_resid_pre` and report three quality metrics:

- **sparsity** $L_0 = \mathbb{E}\|f(x)\|_0$, the number of active features per token;
- **explained variance** $1 - \text{FVU}$, where $\text{FVU} = \sum_t \|x_t - \hat{x}_t\|^2 / \sum_t \|x_t - \bar{x}\|^2$;
- **spliced CE loss**: next-token loss with $\hat{x}$ substituted for x in the forward pass. This is the metric that matters, since a reconstruction can have low MSE and still destroy information the model uses.

	prompt	L_0	expl. var.	CE clean → spliced
1	The Golden Gate Bridge is a suspension bridge in San Francisco.	51.8	0.865	2.68 → 2.83
2	def fibonacci(n): ↵ if n < 2: ↵ return n	58.7	0.646	3.42 → 4.20
3	She was furious, slammed the door, and refused to speak to anyone.	59.3	0.828	3.18 → 3.39

Table 1: Reconstruction quality of the layer-8 residual SAE on three prompts (BOS excluded).

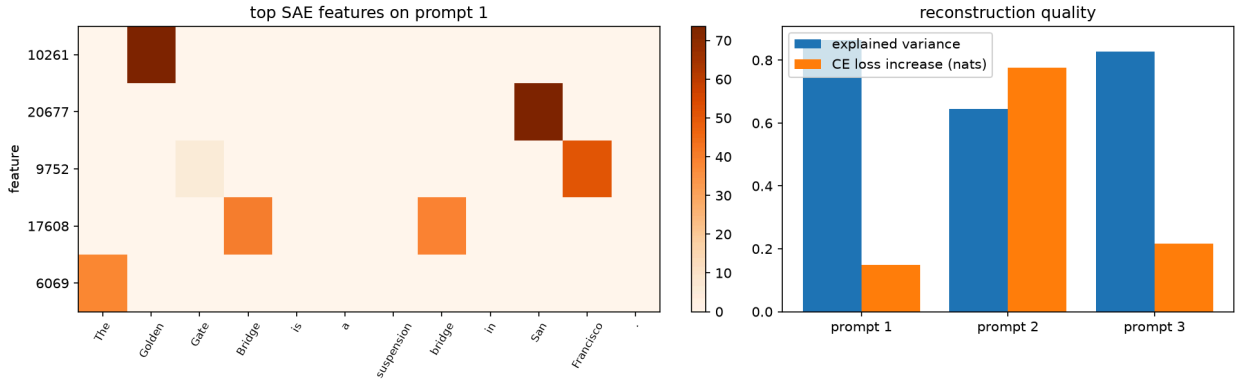


Figure 2: Left: activations of prompt 1's five strongest features across its tokens. Each fires on essentially one token. Right: explained variance and CE-loss increase per prompt.

Finding

On prose the SAE is good: $L_0 \approx 52$ out of 24576, 87% variance explained, and splicing it in costs 0.15 nats. On code it is markedly worse (65% explained, +0.78 nats), which suggests code is underrepresented in its training distribution (OpenWebText). The strongest feature on the code prompt fires on a whitespace token, most likely indentation.

The top features on prompt 1 fire on exactly the tokens you would guess: #10261 (`Golden`), #20677 (`San`), #9752 (`Francisco`), #17608 (`Bridge`). Hand-annotating five features, with a hypothesis formed **before** reading Neuronpedia's label and then stress-tested with your own prompts, is part of card 1.3 and lives in `notes/sae_features_pretrained.md`.

5 Status of §1

DONE TransformerLens + CircuitsVis first pass, logit lens, residual reconstruction.

IN PROGRESS nnsight: local tracing and intervention done. Remote NDIF run pending an API key.

IN PROGRESS SAELens: loading, metrics and feature listing done. Hand-annotation of 5 features (your notes) to do.

TO DO YSDA week 11: probing, then the mech-interp lab (`experiments/00_setup/04_ysda_interpretability.md`).